COM361 Day 7

FORWARD STATE SPACE PLANNING

Forward State Space Planning example:-"Weather forecasting system"

- So now, our fact (b) "Air is very heavy"

has matched with the 'IF' part of the Rule III.

- Thus our new fact will become
  - e) "there is humidity in the air"
  - d) "we suspect temperature is less than 20◦"

- Now, this both new fact e and d are matched with the 'If...AND' part of Rule I.

- Thus our new fact will be

f)"there are chances of rain" Which is nothing but our goal state.

BACKWARD STATE SPACE PLANNING

What is BSSP or Regression Planner In AI?

- Backward state space search or BSSP is also called as regression planner, from the name of this method you can make out that the processing will start from the finishing state and then you will go backwards to the initial state.

- So, basically we try to backward the scenario and find out the best possibility, in order to achieve the goal , to achieve this we have to see what might have been correct action at previous state.

- In forward state space search we need information about the successor of the current state now, for backward state space search we will need information about the predecessor of the current state.

BACKWARD STATE SPACE PLANNING

Backward chaining example:-

- So the statement "there are chances of rain" is present in the 'Then' part of Rule I.

- Thus we get our two sub goals

1. "we suspect temperature is less than 20◦"
2. "there is humidity in the air"

- Now sub goal number 1 is present in the 'Then' part of Rule II.
- Thus we get our another two sub goals

3."Sun is behind the clouds"
4. "air is very cool". Which is nothing but our first and third fact.(a and

BACKWARD STATE SPACE PLANNING

Backward chaining example:-

- And sub goal number 2 is present in the 'Then' part of Rule III. Thus we get our another sub goal

5. "Air is very heavy." Which is nothing but our second fact(b).

Hence we conclude that "there are chances of rain" because we rich all facts(data).

Well Defined Problem

- If a problem has been defined with five component, it is considered a well defined problem
  - initial state, actions, transition model, goal test, path cost.
- INITIAL STATE: The initial state that the agent starts in
- ACTIONS: A description of the possible actions available to the agent.
  - Given a particular state s, ACTIONS(s) returns the set of actions that can be executed in s.
  - Each of these actions is applicable in s.

Well Defined Problem Cont…

- TRANSITION MODEL: A description of what each action does is known as the transition model
  - A function RESULT(s,a) that returns the state that results from doing action a in state s.
  - The term successor to refer to any state reachable from a given state by a single
  - The state space of the problem is the set of all states reachable from the initial state by any sequence
  - of actions.
  - The state space forms a graph in which the nodes are states and the links between nodes are actions.
  - A path in the state space is a sequence of states connected by a sequence of actions.

Well Defined Problem cont…

- GOAL TEST: The goal test determines whether a given state is a goal state.
- PATH COST: A path cost function that assigns a numeric cost to each path.
  - The problem-solving agent chooses a cost function that reflects its own performance measure.
  - The cost of a path can be described as the sum of the costs of the individual actions along the path.
  - The step cost of taking action a in state s to reach state s′ is denoted by c(s, a, s′).
  - A SOLUTION to a problem is an action sequence that leads from the initial state to a goal state.
- Solution quality is measured by the path cost function, and an OPTIMAL SOLUTION has the lowest path cost among all solutions.

Constraint Satisfaction Problem(CSP)

- Refers to problem where goal is to find solution that satisfies a set of constraints.
- Constraint involves variable that certain domain(set of possible values)
- Solucion must assign values to these variable in such a way that all constraint are met
- Variable = elements of the problem
- Constraints = limitation or condition that must be satisfied
- The goal is to find an assignment of values to the variables that satisfies all constraints simultaneously

CSP cont…

- A constraint satisfaction problem consists of three components, X, D, and C
- X is a set of variables, {X1,...,Xn}
- D is a set of domains, {D1,...,Dn}, one for each variable.
- C is a set of constraints that specify allowable combinations of values

CSP cont…(Class reading)

Each constraint C consists of a pair **scope, rel,** where scope is a tuple of variables that participate in the constraint and rel is a relation that defines the values that those variables can take on.

- **Scope:** This is a tuple of variables that participate in the constraint. It specifies which variables are involved in the constraint.
- **Relation:** This defines the allowed combinations of values for the variables in the scope. It specifies the values those variables can take on while satisfying the constraint.

CONSTRAINT SATISFACTION PROBLEMS

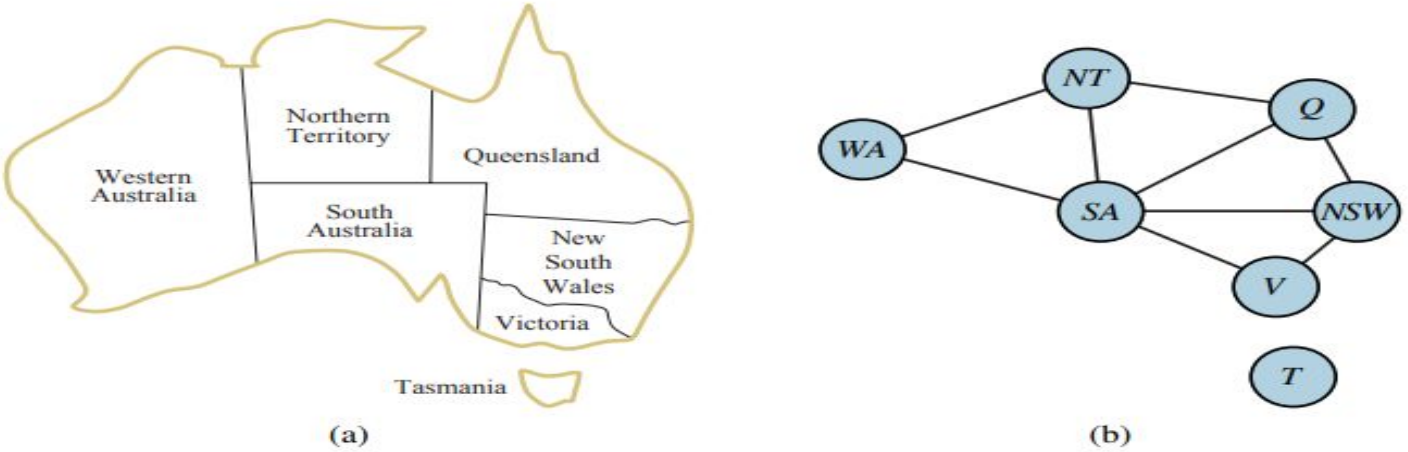


Figure 6.1 (a) The principal states and territories of Australia. Coloring this map can be viewed as a constraint satisfaction problem (CSP). The goal is to assign colors to each region so that no neighboring regions have the same color. (b) The map-coloring problem represented as a constraint graph.

CSP cont…

Variables(X): or V(values)

- map represents a variable in the CSP. Ex., Western Australia, Northern Territory, Queensland, etc. Each variable represents a region that needs to be assigned a color.

Domains(D)

- The domain of each variable represents the set of colors that can be assigned to the corresponding region. Typically, this is a fixed set of colors. For example, {red, green, blue, yellow}.

Constraints(C):

- The constraint in this problem arises from the requirement that no two adjacent regions can have the same color. This constraint is applied to pairs of adjacent territories.
- For each pair of adjacent territories, there is a constraint that the colors assigned to them must be different.

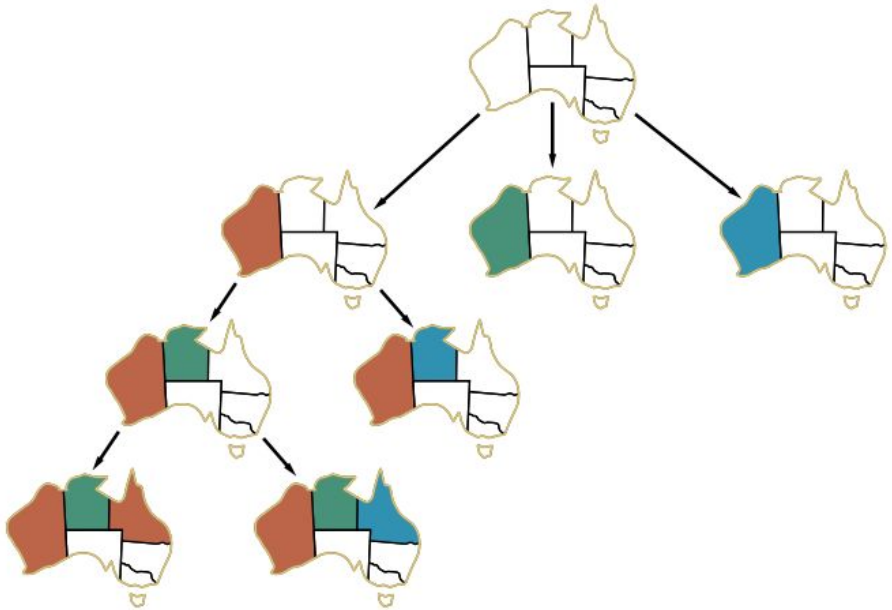


Figure 6.6 Part of the search tree for the map-coloring problem in Figure **??**.

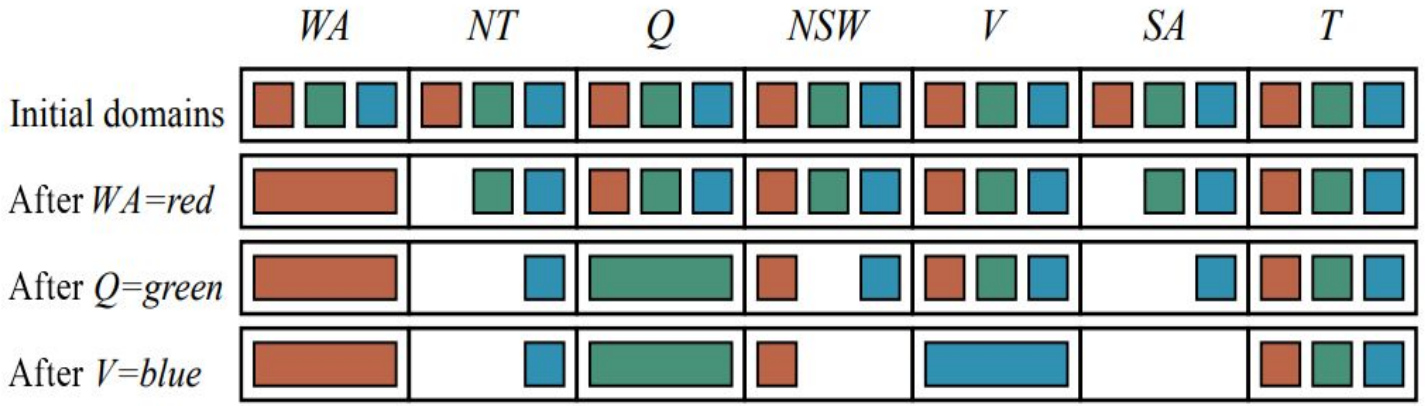
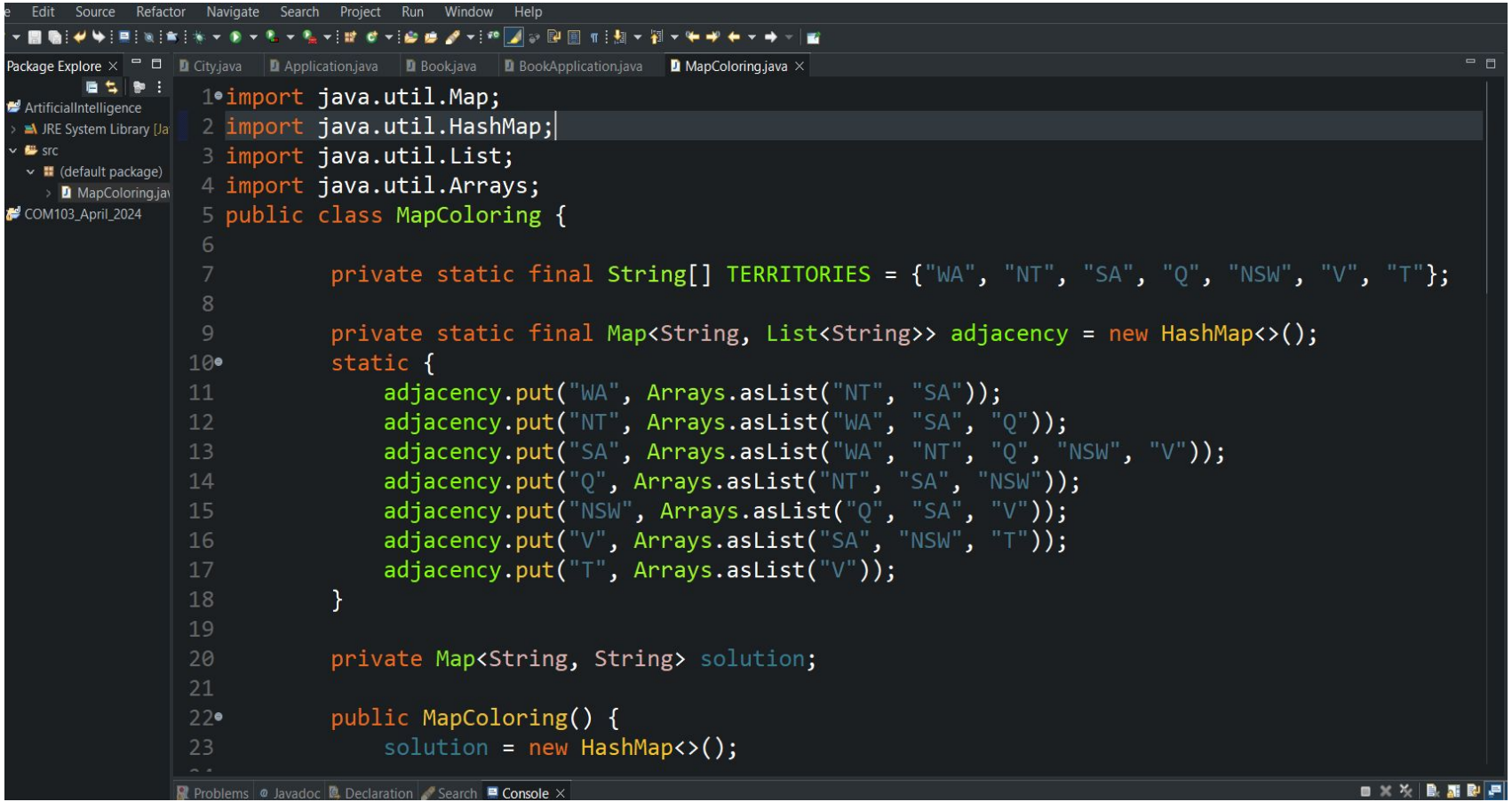


Figure 6.7 The progress of a map-coloring search with forward checking. $WA = red$ is assigned first; then forward checking deletes *red* from the domains of the neighboring variables NT and SA. After $Q = green$ is assigned, *green* is deleted from the domains of NT, SA, and NSW. After $V = blue$ is assigned, *blue* is deleted from the domains of NSW and SA, leaving SA with no legal values.


Edit Source Refactor Navigate Search Project Run Window Help
Package Explore
ArtificialIntelligence
JRE System Library [Ja
src
(default package)
MapColoring.jav
COM103_April_2024
City.java
Application.java
Book.java
BookApplication.java
MapColoring.java
1 import java.util.Map;
2 import java.util.HashMap;
3 import java.util.List;
4 import java.util.Arrays;
5 public class MapColoring {
6
7 private static final String[] TERRITORIES = {"WA", "NT", "SA", "Q", "NSW", "V", "T"};
8
9 private static final Map<String, List<String>> adjacency = new HashMap<>();
10 static {
11 adjacency.put("WA", Arrays.asList("NT", "SA"));
12 adjacency.put("NT", Arrays.asList("WA", "SA", "Q"));
13 adjacency.put("SA", Arrays.asList("WA", "NT", "Q", "NSW", "V"));
14 adjacency.put("Q", Arrays.asList("NT", "SA", "NSW"));
15 adjacency.put("NSW", Arrays.asList("Q", "SA", "V"));
16 adjacency.put("V", Arrays.asList("SA", "NSW", "T"));
17 adjacency.put("T", Arrays.asList("V"));
18 }
19
20 private Map<String, String> solution;
21
22 public MapColoring() {
23 solution = new HashMap<>();
Problems Javadoc Declaration Search Console

```
import java.util.Map;
import java.util.HashMap;
import java.util.List;
import java.util.Arrays;
public class MapColoring {
        private static final String[] TERRITORIES = {"WA", "NT", "SA", "Q", "NSW",
"V", "T"};

        private static final Map<String, List<String>> adjacency = new
HashMap<>();
        static {
        adjacency.put("WA", Arrays.asList("NT", "SA"));
        adjacency.put("NT", Arrays.asList("WA", "SA", "Q"));
        adjacency.put("SA", Arrays.asList("WA", "NT", "Q", "NSW", "V"));
        adjacency.put("Q", Arrays.asList("NT", "SA", "NSW"));
        adjacency.put("NSW", Arrays.asList("Q", "SA", "V"));
        adjacency.put("V", Arrays.asList("SA", "NSW", "T"));
        adjacency.put("T", Arrays.asList("V"));
        }

        private Map<String, String> solution;

        public MapColoring() {
        solution = new HashMap<>();
        }

        private boolean isSafe(String territory, String color) {
        for (String neighbor : adjacency.get(territory)) {
        if (solution.containsKey(neighbor) && solution.get(neighbor).equals(color))
{
        return false; // Color conflict with adjacent territory
        }
        }
        return true;
        }

        private boolean solveMapColoringUtil(int index, String[] colors) {
                if (index == TERRITORIES.length) {
                return true; // All territories are colored
                }
                String territory = TERRITORIES[index];
                for (String color : colors) {
                if (isSafe(territory, color)) {
                solution.put(territory, color);
                if (solveMapColoringUtil(index + 1, colors)) {
                        return true; // Solution found
                }
                // Backtrack
                solution.remove(territory);
                }
                }
                return false; // No valid coloring found
                }
                public Map<String, String> solveMapColoring(String[] colors) {
                if (solveMapColoringUtil(0, colors)) {
                return solution;
                } else {
                return null; // No solution exists
                }
                }
                public static void main(String[] args) {
                MapColoring mapColoring = new MapColoring();
                String[] colors = {"Red", "Green", "Blue", "Yellow"};
                Map<String, String> solution =
mapColoring.solveMapColoring(colors);
                if (solution != null) {
                for (Map.Entry<String, String> entry : solution.entrySet()) {
                System.out.println(entry.getKey() + ": " + entry.getValue());
                }
                } else {
                System.out.println("No solution found.");
                }
                }
    }
```